

Analysis: Scheduling a Meeting

Test Set 1

In Test Set 1, we can simply try all possible options to schedule a new meeting. We can try all possible start times for a new meeting, for each Tech Lead calculate how many of their scheduled meetings overlap with our proposed meeting time, and then pick **K** smallest of these values. Output the minimum for all the possible start times as the final answer.

This can be done in $O(\mathbf{D} \times (\mathbf{M} + \mathbf{N} \log \mathbf{N}))$ time complexity, or similar, which is fast enough for this test set.

Note that limits in Test Set 1 are very permissive, and thus it is possible to pass it with even slower solutions. For example, we can try all possible starting times and all possible subsets of Tech Leads who will attend the meeting, and for each such option count how many already scheduled meetings overlap with the given time slot and have to be canceled. This will result in a solution with $O(\mathbf{D} \times 2^{\mathbf{N}} \times \mathbf{M})$ overall time complexity.

Test Set 2

In Test Set 2, the limits are higher, so we will need a smarter solution.

There are many possible ways to solve this problem, and we list some of them below. All approaches in this analysis use the following techniques and observations:

1. We will again try all possible start times for a new meeting, but now in a sliding window fashion. This means that first we will try the range of $(0, \mathbf{X})$ hours, then $(1, \mathbf{X} + 1)$, and so on.
2. We will maintain the number of current meetings for each Tech Lead as meetings enter and exit our sliding window.
3. To minimize the number of cancelled meetings we need to meet with **K** Tech Leads having the smallest number of current meetings. Sum of their numbers of meetings will be the answer for the current window.

Approach #1. Data structures and binary search.

Let us maintain two helper arrays:

- Number of Tech Leads that have exactly i meetings in the current window, for each i . Note that if you take a prefix sum $SUM(0, j)$ over this array, you will find the total number of people with less than or equal to j meetings for some j .
- Sum of number of meetings for everyone with exactly i meetings in the current window, for all i . Note that if you take a prefix sum $SUM(0, j)$ over this array, you will find the total number of meetings for everyone with less than or equal to j meetings for some j .

To be able to quickly take prefix sums over these arrays, we will keep the values in a data structure like segment tree or [Fenwick tree](#).

Armed with these data structures and helper arrays, in each window we can use binary search to find the biggest number of meetings among **K** Tech Leads we will meet with, and then the sum of the number of their meetings.

Time complexity of this solution will be $O(\mathbf{D} + \mathbf{N} + \mathbf{M} \log \mathbf{M} + \mathbf{D} \log^2 \mathbf{M})$ (which you can also optimize to $O(\mathbf{D} + \mathbf{N} + \mathbf{M} \log \mathbf{M} + \mathbf{D} \log \mathbf{M})$ if you replace binary search with some kind of smart segment tree descent).

Approach #2. Rebalancing sets.

Another elegant solution is to store the number of the current meetings for each Tech Lead in a data structure that allows to extract the minimum and the maximum value, and remove an arbitrary value from it, like a tree-based set. We will keep two of these data structures: one containing $\mathbf{K}$ Tech Leads with the minimum number of meetings in the current window, and another with $\mathbf{N} - \mathbf{K}$ other Tech Leads. We will update these data structures as the meetings come and go: as a meeting starts or ends, we remove an old entry for the corresponding Tech Lead in the data structure, insert a new entry with an updated meetings count, and rebalance two data structures so that one of them again has $\mathbf{K}$ smallest values (to achieve this, we may need to exchange the biggest value from one data structure with the smallest value in the second one). In addition, we will also maintain the sum of the number of meetings in the data structure that holds $\mathbf{K}$ smallest values.

It can be noted that with careful implementation each meeting will trigger only a constant number of operations to keep two data structures balanced, and the total time complexity of this solution will be $O(\mathbf{D} + \mathbf{N} + \mathbf{M} \log \mathbf{N})$.

Approach #3. Linear solution.

Finally, the "linear" $O(\max(\mathbf{D}, \mathbf{N}, \mathbf{M}))$ solution to this problem exists. This is possible if you carefully maintain the following values:

- Number of the current meetings for each Tech Lead.
- Number of Tech Leads with exactly i current meetings for all i .
- Value of maximum number of meetings among $\mathbf{K}$ Tech Leads we will meet with.
- Number of Tech Leads with that maximum number of meetings among $\mathbf{K}$ Tech Leads we will meet with.
- Sum of the meetings of all $\mathbf{K}$ Tech Leads we will meet with (which is also the sum of $\mathbf{K}$ smallest numbers of current meetings).

It can be noted that with each starting or ending meeting, all these values can be recalculated in $O(1)$ time (in fact, most of these values change at most by one), and we can achieve a solution with total linear time complexity.

Analysis: Sort the Fabrics

Let us denote the fabric array sorted by Ada as Ad , and the one sorted by Charles as Ch .

Test Set 1

The maximum number of fabrics here is 2.

One fabric

When there's only one fabric, the answer is always 1.

Two fabrics

When there are two fabrics, the answer is either 0 or 2.

If the two fabrics have the same color and the same durability, the sort will be done by the unique identifier, leading to the same ordering of the fabrics in both Ad and Ch ; hence the answer is 2.

If the two fabrics have the same color, then Ad would be sorted in the ascending order of the unique identifier. We need to check, if the smaller unique index has the smaller durability of the two fabrics or not. If yes, then the fabrics in Ch will be in the same order as Ad , and the answer is 2; otherwise the answer is 0.

If the two fabrics have the same durability, then Ch would be sorted in the ascending order of the unique identifier. We need to check, if the smaller unique index has the lexicographically smaller color of the two fabrics, or not. If yes, then the fabrics in Ad will be in the same order as Ch , and the answer is 2; otherwise the answer is 0.

If the two fabrics have different colors and durabilities and if the less durable fabric also has the lexicographically smaller color, then in both Ad and Ch , the sorted array will have the same order of fabrics, and the answer will be 2. If that is not the case, the answer is 0.

In all of the approaches discussed above, the time complexity is $O(1)$, which is sufficient for Test Set 1.

Test Set 2

We can create two instances of the fabric array, and sort one of them to create Ad , and another one to create Ch . Now, we can compare the two arrays, and the number of fabrics that have the same index in both of them is the required answer.

Here, the time complexity of sorting by ascending durability is $O(N \log N)$, and the time complexity of sorting by ascending color is $O(N \log N \times \max |C_i|)$. Running a loop over Ad and Ch is $O(N)$. So, the total time complexity is $O(N \log N \times \max |C_i|)$, which is sufficient for Test Set 2.

Analysis: Story of Seasons

For each type of seeds i , let us use $deadline_i$ to represent the latest day it should be planted to earn profits, where $deadline_i$ is calculated as

$$deadline_i = D - L_i.$$

Which means if one seed of type i is planted on or before $deadline_i$, it can mature and let us earn V_i eventually. Otherwise it cannot. In other words, for each type i , if the current day is already later than $deadline_i$, then all seeds of type i are not worthy to plant anymore.

Test Set 1

Since each vegetable has one seed and we can plant only one seed per day, we should plant one seed everyday from the day one, until there is no seed available or worth to plant. Since N is very small in this case, we can use brute force search with dynamic programming to implement it.

We can maintain a set S that contains the types of seeds that are not planted yet. And on each day d , we will try to plant every type i from S . If $deadline_i \leq d$, then we can earn V_i from this day's work, otherwise we earn 0. And then we go on to the next day d with remaining types $S - \{i\}$ to see what maximum profit we can earn. Formally, we have the recursive function as below:

$$Profit(d, S) = MAX_{i \in S} (Earn(d, i) + Profit(d + 1, S - \{i\})),$$

where $Earn(d, i) = V_i$ if $d \leq deadline_i$ otherwise $Earn(d, i) = 0$. Note that if d is larger than D or if S is empty, then $Profit(d, S)$ should directly return 0. Then $Profit(1, \{1, \dots, N\})$ is the answer of the problem.

We can use dynamic programming for the search. That is, prepare a table to store the answer of every state (d, S) . Note that the value of d is actually implied by the associated S that $d = N - |S| + 1$. Therefore the total number of states equals to the number of combinations of S , which is $O(2^N)$. And in each searching state, we iterate through all the available types in S , which is an $O(N)$ operation. Therefore, the time complexity of this solution is $O(N \times 2^N)$.

Test Set 2

Instead of searching for all possible combinations, we can use greedy strategy on this problem. For simplicity, let us still use the limits that $X = 1$ and $Q_i = 1$ for all i .

We might not know which type to plant on day 1 in order to achieve the optimal solution, but we can find out which type to plant on day $D - 1$. We just need to gather all types that have $deadline_i \geq D - 1$ and choose the one with the maximum V_i to plant on day $D - 1$.

Let us prove that this is always the best choice. Say type g is the one we select for day $D - 1$ in our greedy strategy, we need to prove that for any other schedule that does not plant type g on day $D - 1$, we can always achieve better or the same profit by modifying it to plant g on day $D - 1$:

- If a schedule does nothing on day $D - 1$, and the seed g does not exist in this schedule: We can put g in day $D - 1$ of this schedule to increase the total profit by V_g .

- If a schedule does nothing on day $\mathbf{D} - 1$, but seed g is scheduled to be planted on some other day d : We can move g from day d to day $\mathbf{D} - 1$ of this schedule and the total profit will stay the same.
- If a schedule plant other seed o on day $\mathbf{D} - 1$, and the seed g does not exist in this schedule: We can replace seed o with seed g on day $\mathbf{D} - 1$ to increase the total profit by $V_g - V_o$. Since g is the one with maximum value among the non-expired seeds in day $\mathbf{D} - 1$, we have that $V_g - V_o \geq 0$.
- If a schedule plant other seed o on day $\mathbf{D} - 1$, but the seed g is scheduled to be planted on some other day d : We can exchange seed o and seed g so that o is planted on day d and g is planted on day $\mathbf{D} - 1$, and the total profit will stay the same.

Therefore, we are sure that planting the type with maximum V among all non-expired types of day $\mathbf{D} - 1$ is always the best. And for day $\mathbf{D} - 2, \mathbf{D} - 3, \dots$, to day 1, we do the same for each of them because they can be seen as a smaller problem of the original one.

To implement this, we can sort the types by their deadline, and maintain a max-heap to store the types with their values. Then we iterate from day $\mathbf{D} - 1$ to day 1, put all the types whose deadline is equal to the current day into the max-heap, and take the one with the maximum value from the heap to assign for the current day.

This strategy can also be used on the case that $\mathbf{X} > 1$ and $\mathbf{Q}_i > 1$. For each day with $\mathbf{X}$ slots, we first insert all types that has deadline equal to the current day, then repeatedly take one type from the heap to assign for the current day until the heap is empty or the $\mathbf{X}$ slots are filled.

In this implementation, we insert each type to the heap on the days equal to their deadline, so we totally insert the heap $O(\mathbf{N})$ times. We pop out each type whenever they are used up, so we totally pop the heap $O(\mathbf{N})$ times. We access the top from the heap on each new day and whenever a previous type is used up, so the number of times we access the top of the heap is $O(\mathbf{D} + \mathbf{N})$. Therefore the total time complexity we spend on operating the heap is $O((\mathbf{D} + \mathbf{N}) \times \log \mathbf{N})$. And we spent $O(\mathbf{N} \times \log \mathbf{N})$ time on sorting the types by their deadline. Therefore the total time complexity of this implementation is $O((\mathbf{D} + \mathbf{N}) \times \log \mathbf{N})$.

Test Set 3

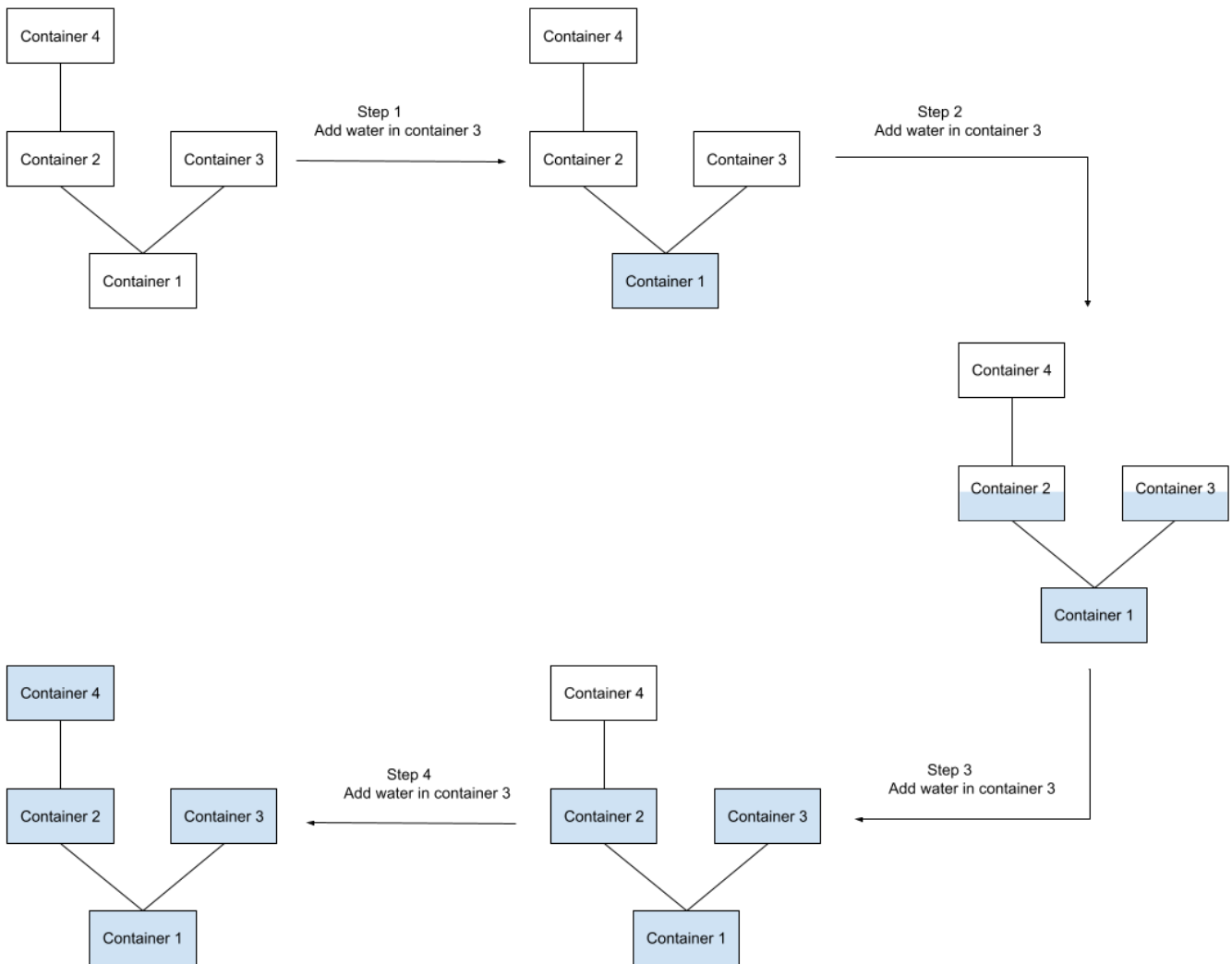
In the implementation of Test Set 2, we iterate through all $\mathbf{D}$ days to fill the slots. In this case with $\mathbf{D}$ up to 10^{12} , we are not able to iterate through all of them. However, only the days that new types are added to the heap need our attention, otherwise every day is just some slots to fill.

Therefore, we can divide these $\mathbf{D}$ days into several segments, where the end of each segment is the deadline of some types. Then the number of slots of each segment is the length of the segment times $\mathbf{X}$. Then we iterate through the segments from the latest one, put all types with associated deadline into the heap, and take the most expensive ones from the heap to fill the slots of the current segment.

In this implementation, the number of segments is $O(\mathbf{N})$. The number of times we insert and pop the heap is the same as the implementation of Test Set 2, which is $O(\mathbf{N})$. And we access the top of the heap when we process each new segment and whenever a previous type is used up, so the number of time we access the top is $O(\mathbf{N})$. Therefore the total time complexity we spend on operating the heap is $O(\mathbf{N} \times \log \mathbf{N})$. With the $O(\mathbf{N} \times \log \mathbf{N})$ time we spent on sorting the deadlines and $O(\mathbf{N})$ time we spent on building the segments, the total time complexity of this solution is $O(\mathbf{N} \times \log \mathbf{N})$.

Analysis: Water Container System

Because of the [Communicating vessels](#) principle, all containers in the same level are filled up equally, and containers of level i will start filling up only when all containers of level $i - 1$ are full. This principle is also evident from the example in the problem statement, which we repeat here. The first liter of water added flows down to container 1 filling it up completely. Since the first level consisting of just the root container is now filled completely, the second liter of water starts filling both containers of the second level equally. The same effect would have been achieved if we were to add water to any container other than container 3. The third liter of water fills up the second level completely, and only then the sole container in the third level is filled up with the fourth and final liter of water.



As we can see, it does not even matter, which containers are used for adding more water to the system. All we need to know is the final volume of water in the system, which is Q liters, and the number of containers C_i in each level. Then the number of containers at level i or below is $S_i = \sum_{j=1}^i C_j$, and the answer is the largest S_i such that $S_i \leq Q$.

Test Set 1

Since we are dealing with a perfect binary tree in this test set, $C_i = 2^{i-1}$, and the solution boils down to the following simple loop. We just add powers of 2 to the answer as long as it does not

exceed the limit Q .

```
int solve(int Q) {
    int ans = 0;
    int C = 1;
    while (ans + C <= Q) {
        ans += C;
        C *= 2;
    }
    return ans;
}
```

The time complexity of this solution is proportional to the depth of the tree, which is $O(\log N)$.

Test Set 2

Here we have an arbitrary rooted tree, so we must make the extra effort to count the number of containers C_i in each level using any tree traversal algorithm, for example, [Depth-first search](#). Otherwise, the logic remains the same.

```
int solve(int Q, vector<int> C) {
    int ans = 0;
    int level = 1;
    while (level < C.size() && ans + C[level] <= Q) {
        ans += C[level];
        level++;
    }
    return ans;
}
```

The time complexity of this solution is $O(N)$ because of the Depth-first search.